

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение высшего образования
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ»

Кафедра _____
(наименование)

ОТЧЁТ ПО ПРАКТИКЕ
ЗАЩИЩЁН С ОЦЕНКОЙ
РУКОВОДИТЕЛЬ

ст. преподаватель		М. Д. Поляк
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЁТ ПО ПРАКТИКЕ

вид практики	производственная
тип практики	Технологическая(проектно-технологическая)
на тему индивидуального задания	Фреймворк для вычислительных экспериментов с АКАР-управлением

выполнен	Сергеевой Еленой Александровной
	фамилия, имя, отчество обучающегося в творительном падеже

по направлению подготовки	09.03.04	Программная инженерия
	код	наименование направления

	наименование направления	
направленности	02	Проектирование программных систем
	код	наименование направленности

	наименование направленности
--	-----------------------------

Обучающийся группы №	4331	Е. А. Сергеева
	номер	подпись, дата
		инициалы, фамилия

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
1. Задание	4
1.1 Обязательная часть	4
1.2 Необязательная часть, уровень 1 (+1 балл)	5
1.3 Необязательная часть, уровень 2 (+2 балла)	5
2. Описание задачи и методов решения.....	6
2.1 Модель хищник–жертва (Лотка–Вольтерра)	6
2.2 Метод АКАР — основная идея	6
2.3 Алгоритм синтеза (тривиальный случай).....	6
2.4 Пример: аддитивное управление по x , $\psi = x - x^*$	7
2.5 Пример: мультипликативное управление по x , $\psi = x - x^*$	7
2.6 Метрики качества управления	8
3. Реализация	10
3.1 Архитектура фреймворка	10
3.2 Метрики качества.....	12
3.3 Работа фреймворка demo.jrpnb	12
ЗАКЛЮЧЕНИЕ.....	28
ПРИЛОЖЕНИЕ.....	30

ВВЕДЕНИЕ

Математическое моделирование позволяет изучать поведение различных систем без проведения реальных экспериментов. С его помощью можно исследовать изменение состояния системы во времени, оценивать влияние различных параметров и проверять работу методов управления. Одной из наиболее известных моделей такого типа является модель Лотки–Вольтерра, которая описывает взаимодействие популяций жертв и хищников.

В данной работе рассматривается применение метода АКАР (аналитического конструирования агрегированных регуляторов) для управления моделью Лотки–Вольтерра. Этот метод позволяет построить закон управления, который переводит систему в заданное состояние и удерживает ее вблизи точки равновесия.

Цель работы — разработать программный фреймворк на языке Python для проведения вычислительных экспериментов с моделью Лотки–Вольтерра. Фреймворк должен позволять запускать эксперименты с разными параметрами, сохранять результаты, повторно воспроизводить их, сравнивать несколько экспериментов между собой и строить графики.

В ходе работы были реализованы основные модули фреймворка, вычисление показателей качества управления, сохранение результатов в файлы JSON и CSV, построение графиков и сводных таблиц. Также была добавлена модель Лотки–Вольтерра с внутривидовой конкуренцией, что показывает возможность расширения фреймворка. Дополнительно реализован символьный вывод закона управления с использованием библиотеки SymPy. Упрощённое управление зависимостями (установка библиотек через pip/conda);

1. Задание

1.1 Обязательная часть

Реализовать все методы помеченные TODO в файле `framework_skeleton.py`. Все тесты в разделе 7 скелета должны пройти без ошибок.

Требования к реализации:

1. `compute_control`: реализовать оба типа управления — `additive` и `multiplicative`.
2. `make_rhs`: возвращать функцию `rhs(t, state)` пригодную для передачи в `solve_ivp`.
3. `compute_metrics`: вычислять все четыре метрики (τ , $\max|\psi|$, $\max|u|$, `e_final`). При `sol.success==False` возвращать `inf`.
4. `save_experiment`: сохранять `config` и `metrics` в JSON. Массивы не сохранять.
5. `load_and_reproduce`: загружать JSON и вызывать `run_experiment` с восстановленной конфигурацией.
6. `save_metrics_csv`: сохранять метрики списка экспериментов в CSV с заголовком.
7. `plot_experiment`: четыре графика на одной фигуре 2×2 (x/y , u , ψ , фазовый портрет). Подписи осей обязательны.
8. `plot_compare`: несколько кривых на одном графике, каждая подписана именем эксперимента.
9. `summary_table`: вывести pandas DataFrame с колонками `name`, T , τ , $\max|\psi|$, $\max|u|$, `e_final`.

Требование к расширяемости:

Добавление новой модели должно требовать изменений только в двух местах: новый метод `equilibrium()` в `ModelParams` и новая реализация `make_rhs`. Всё остальное должно работать без изменений. Продемонстрируйте это: добавьте модель с внутривидовой конкуренцией (вариант Б) и запустите для неё один эксперимент.

1.2 Необязательная часть, уровень 1 (+1 балл)

Продemonстрировать работу фреймворка на серии экспериментов в Jupyter Notebook:

1. Запустить перебор по $T \in [0.1, 5.0]$ (10-20 значений) через фреймворк.
2. Сохранить все эксперименты в JSON-файлы и в сводный CSV.
3. Построить `plot_compare` для всех экспериментов по метрике $\psi(t)$.
4. Вывести `summary_table` и прокомментировать: при каком T лучший баланс скорости и стоимости управления?
5. Воспроизвести лучший эксперимент из сохранённого JSON и убедиться что результат совпадает.

1.3 Необязательная часть, уровень 2 (+2 балла)

Реализовать модуль символьного вывода управления на основе sympy:

Что должен делать модуль:

1. Принимать на вход: символьные уравнения модели $\dot{x} = f(x,y)$, $\dot{y} = g(x,y,u)$, макропеременную $\psi(x,y)$, параметр T .
2. Вычислять $\dot{\psi} = \partial\psi/\partial x \cdot f + \partial\psi/\partial y \cdot g$ через `sympy.diff` и `sympy.simplify`.
3. Решать уравнение $\dot{\psi} = -\psi/T$ относительно u через `sympy.solve`.
4. Возвращать: символьное выражение $u(x,y)$ и лямбда-функцию для численного вычисления.

Проверка:

5. Применить модуль к классической модели с $\psi = x - x^*$ (аддитивное управление по x).
6. Сравнить символьный результат с аналитическим из задания `adar_assignment.docx` — они должны совпасть.
7. Применить к той же модели с $\psi = x/y - c^*$ — вывести управление символьно.

2. Описание задачи и методов решения

2.1 Модель хищник–жертва (Лотка–Вольтерра)

Классическая модель описывает динамику двух взаимодействующих популяций — жертвы $x(t)$ и хищника $y(t)$:

$$\dot{x} = \alpha x - \beta xy$$

$$\dot{y} = \delta xy - \gamma y,$$

где α — скорость роста жертвы, β — интенсивность выедания, δ — эффективность хищника, γ — смертность хищника. Все параметры положительны.

Ненулевое равновесие (точка, где $\dot{x} = 0$ и $\dot{y} = 0$):

$$x^* = \gamma/\delta, \quad y^* = \alpha/\beta$$

2.2 Метод АКАР — основная идея

Метод аналитического конструирования агрегированных регуляторов (АКАР) позволяет синтезировать управляющее воздействие $u(t)$, которое переводит систему из произвольного начального состояния в заданное целевое.

Вместо управления каждой переменной отдельно вводится макропеременная ψ — скалярная функция состояния, равная нулю в целевом состоянии. Управление строится так, чтобы ψ экспоненциально убывала к нулю.

Интуиция: если $\psi = x - x^*$, то $\psi = 0$ означает $x = x^*$. Задача — найти u такое, чтобы $\psi \rightarrow 0$ при $t \rightarrow \infty$.

2.3 Алгоритм синтеза (тривиальный случай)

Тривиальный случай: управление u входит в то же уравнение, что и переменная, по которой задана макропеременная.

1. Задать макропеременную, равную нулю в целевом состоянии:

$$\psi = x - x^*$$

2. Задать желаемую динамику — экспоненциальное затухание с параметром $T > 0$ (уравнение Эйлера-Лагранжа):

$$\dot{\psi} = -\psi/T$$

Это гарантирует $\psi(t) = \psi(0) \cdot \exp(-t/T) \rightarrow 0$. Чем меньше T , тем быстрее сходимость.

3. Вычислить $\dot{\psi}$ через модель, подставив правые части уравнений:

$$\dot{\psi} = d\psi/dt = \partial\psi/\partial x \cdot \dot{x} + \partial\psi/\partial y \cdot \dot{y}$$

4. Приравнять к желаемой динамике и выразить u :

$$\partial\psi/\partial x \cdot \dot{x}(x,y,u) + \partial\psi/\partial y \cdot \dot{y}(x,y,u) = -\psi/T$$

2.4 Пример: аддитивное управление по x , $\psi = x - x^*$

Управление u входит аддитивно в первое уравнение:

$$\dot{x} = \alpha x - \beta xy + u$$

$$\dot{y} = \delta xy - \gamma y$$

Шаг 1. $\psi = x - x^*$, $x^* = \gamma/\delta$

Шаг 2. $\dot{\psi} = -\psi/T$

Шаг 3. Так как $\partial\psi/\partial x = 1$ и $\partial\psi/\partial y = 0$:

$$\dot{\psi} = \dot{x} = \alpha x - \beta xy + u$$

Шаг 4. Приравниваем и выражаем u :

$$\alpha x - \beta xy + u = -(x - x^*)/T$$

$$u = -(x - x^*)/T - \alpha x + \beta xy$$

2.5 Пример: мультипликативное управление по x , $\psi = x - x^*$

Управление u заменяет параметр β — регулирует интенсивность выедания:

$$\dot{x} = \alpha x - u \cdot xy$$

$$\dot{y} = \delta xy - \gamma y$$

Шаг 1–2. $\psi = x - x^*$, $\dot{\psi} = -\psi/T$ (те же, что в аддитивном случае)

Шаг 3. Так как $\partial\psi/\partial x = 1$ и $\partial\psi/\partial y = 0$:

$$\dot{\psi} = \dot{x} = \alpha x - u \cdot xy$$

Шаг 4. Приравниваем и выражаем u :

$$\alpha x - u \cdot xy = -(x - x^*)/T$$

$$u = (\alpha x + (x - x^*)/T) / (xy)$$

2.6 Метрики качества управления

После проведения вычислительного эксперимента необходимо оценить, насколько эффективно система достигла заданного состояния. Для этого используются метрики качества управления. Они позволяют сравнивать различные варианты управления и выбирать наиболее удачные значения параметра регулятора.

Первая метрика — время сходимости τ . Она показывает, за какое время система приблизилась к точке равновесия. В данной работе момент сходимости определяется как первый момент времени, когда значение макропеременной становится меньше 1 % от своего начального значения. Если за время моделирования это условие не выполняется, считается, что система не достигла требуемого состояния.

Вторая метрика — максимальное отклонение ψ . Она показывает наибольшее отклонение системы от точки равновесия за все время моделирования. Чем меньше значение этой метрики, тем стабильнее ведет себя система в процессе перехода к равновесию.

Третья метрика — максимальное управление u . Она характеризует максимальное по модулю значение управляющего воздействия. Эта величина показывает, насколько интенсивное управление потребовалось для достижения цели. Большие значения могут означать высокую стоимость управления или сложность его практической реализации.

Последняя метрика — остаточная ошибка e . Она вычисляется как абсолютное отклонение переменной (x) от равновесного значения в конце моделирования. Эта метрика показывает, насколько точно система достигла требуемого состояния к моменту окончания эксперимента.

Использование всех четырех метрик позволяет оценить работу регулятора с разных сторон. Время сходимости характеризует скорость переходного процесса, максимальное отклонение — качество перехода, максимальное управление — затраты на управление, а остаточная ошибка — точность достижения равновесия. При анализе результатов обычно

выбирается такой параметр регулятора (T), при котором обеспечивается хороший баланс между скоростью сходимости и величиной управляющего воздействия.

3. Реализация

3.1 Архитектура фреймворка

Разработанный фреймворк имеет модульную архитектуру, в которой каждый компонент отвечает за выполнение определенной задачи. Такое разделение упрощает сопровождение программы, позволяет повторно использовать код и облегчает добавление новых моделей и методов управления. Основные модули фреймворка представлены в таблице 3.1.

Таблица 3.1 – Архитектура разработанного фреймворка

Модуль	Назначение
ModelParams	Хранение параметров математической модели и вычисление точки равновесия.
CompetitionModelParams	Реализация модели Лотки–Вольтерра с внутривидовой конкуренцией и вычисление ее равновесия.
ControlParams	Хранение параметров регулятора (параметр (T) и тип управления).
SimParams	Хранение параметров численного моделирования (время моделирования, число точек, точность интегрирования и начальные условия).
ExperimentConfig	Объединение параметров модели, управления и моделирования в единую конфигурацию эксперимента.
compute_control()	Вычисление управляющего воздействия для аддитивного и

	мультипликативного управления.
make_rhs()	Формирование правой части системы дифференциальных уравнений для выбранной модели.
Metrics	Хранение метрик качества управления.
compute_metrics()	Вычисление времени сходимости, максимального отклонения, максимального управления и остаточной ошибки.
run_experiment()	Запуск вычислительного эксперимента, численное решение системы и формирование результата.
ExperimentResult	Хранение результатов моделирования, включая временные ряды и вычисленные метрики.
save_experiment()	Сохранение конфигурации эксперимента и метрик в формате JSON.
load_and_reproduce()	Загрузка конфигурации из JSON и повторное выполнение эксперимента.
save_metrics_csv()	Сохранение метрик нескольких экспериментов в формате CSV.
plot_experiment()	Построение графиков результатов одного эксперимента.
plot_compare()	Сравнение нескольких экспериментов на одном графике.

summary_table()	Формирование сводной таблицы с основными метриками экспериментов.
derive_control_symbolic()	Символьный вывод закона управления с использованием библиотеки SymPy.

3.2 Метрики качества

Для каждого эксперимента вычисляются четыре метрики. Они реализованы в классе Metrics и функции compute_metrics скелета. Описание представлено в таблице 3.2

Таблица 3.2 - Метрики

Обозначение	Название	Определение
tau	Время сходимости	Момент времени когда $ \psi(t) < 0.01 \cdot \psi(0) $. None если не достигнуто за t_max.
max_psi	Макс. отклонение	Максимальное $ \psi(t) = x(t) - x^* $ за всё время моделирования.
max_u	Макс. управление	Максимальное $ u(t) $ — стоимость управления.
e_final	Остаточная ошибка	$ x(t_{end}) - x^* $ — насколько точно система дошла до равновесия к концу времени.

Параметры моделирования: $t_{max} = 100$, $n_{eval} = 10000$, $x_0 = 1.5 \cdot x^*$, $y_0 = 0.5 \cdot y^*$, $rtol = 1e-8$.

3.3 Работа фреймворка demo.jupyter

Код реализации фреймворка представлен в Приложении

✓ Импорт библиотек и фреймворка

```
from framework import *

import numpy as np
import pandas as pd
from pathlib import Path
import sympy as sp
```

✓ Демонстрация работы фреймворка

Для проверки работоспособности разработанного фреймворка выполним один вычислительный эксперимент с использованием параметров модели и параметров численного моделирования, заданных по умолчанию.

В ходе выполнения эксперимента автоматически будут выполнены следующие этапы:

- вычисление координат положения равновесия модели;
- построение замкнутой системы дифференциальных уравнений с законом управления;
- численное интегрирование системы на заданном временном интервале;
- вычисление всех метрик качества управления;
- построение стандартных графиков, характеризующих динамику системы.

```
cfg = ExperimentConfig(
    name="Base experiment"
)

result = run_experiment(cfg)

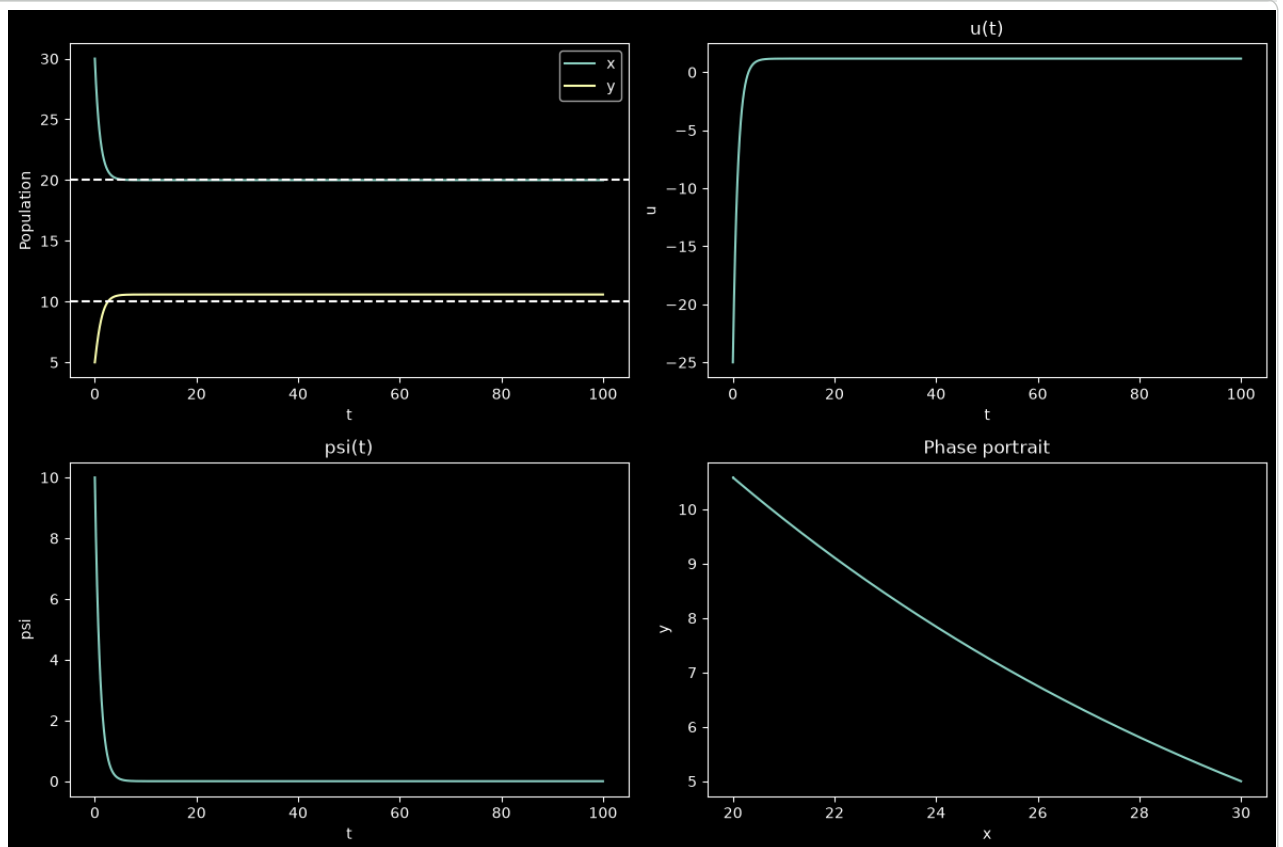
print("Метрики эксперимента:\n")

print(f"Время сходимости ( $\tau$ ):           {result.metrics.tau:.6f}")
print(f"Максимальное  $|\psi|$ :           {result.metrics.max_psi:.6f}")
print(f"Максимальное  $|u|$ :           {result.metrics.max_u:.6f}")
print(f"Остаточная ошибка:           {result.metrics.e_final:.6e}")
```

Метрики эксперимента:

Время сходимости (τ):	4.610461
Максимальное $ \psi $:	10.000000
Максимальное $ u $:	25.000000
Остаточная ошибка:	3.098370e-07

```
plot_experiment(result)
```



Анализ результатов

Система быстро выходит на заданное положение равновесия. Численность жертв уменьшается с 30 до 20 особей, а численность хищников увеличивается с 5 до 10 особей. После этого обе популяции практически не изменяются, что говорит об устойчивости системы.

График ошибки $\psi(t)$ показывает, что отклонение от равновесия быстро уменьшается.

На графике управления видно, что в начальный момент регулятор создаёт достаточно сильное воздействие, чтобы быстро перевести систему в состояние равновесия. Затем величина управления останавливается и остаётся практически постоянной, так как необходимость в дальнейшем интенсивном воздействии исчезает.

Фазовый портрет показывает плавное движение системы к положению равновесия без заметных колебаний, что также подтверждает устойчивость закона управления.

Остаточная ошибка имеет очень малое значение, поэтому к концу моделирования система практически полностью достигает заданного состояния равновесия.

Демонстрация мультипликативного управления

В предыдущем разделе была рассмотрена работа АКАР-регулятора с **аддитивным управлением**. Далее продемонстрируем работу фреймворка для **мультипликативного управления**, при котором управление изменяет коэффициент взаимодействия между популяциями.

После этого сравним оба варианта управления по динамике ошибки, изменению популяций и величине управляющего воздействия.

```
# Конфигурация эксперимента с мультипликативным управлением
```

```
cfg_mul = ExperimentConfig(
    name="multiplicative",
    control=ControlParams(
        T=1.0,
```

```

        control_type="multiplicative"
    )
)

result_mul = run_experiment(cfg_mul)

```

```

print("Метрики эксперимента:\n")
m = result_mul.metrics
print(f"Время сходимости ( $\tau$ ):      {m.tau:10.6f}")
print(f"Максимальное  $|\psi|$ :          {m.max_psi:10.6f}")
print(f"Максимальное  $|u|$ :            {m.max_u:10.6f}")
print(f"Остаточная ошибка:          {m.e_final:10.6e}")

```

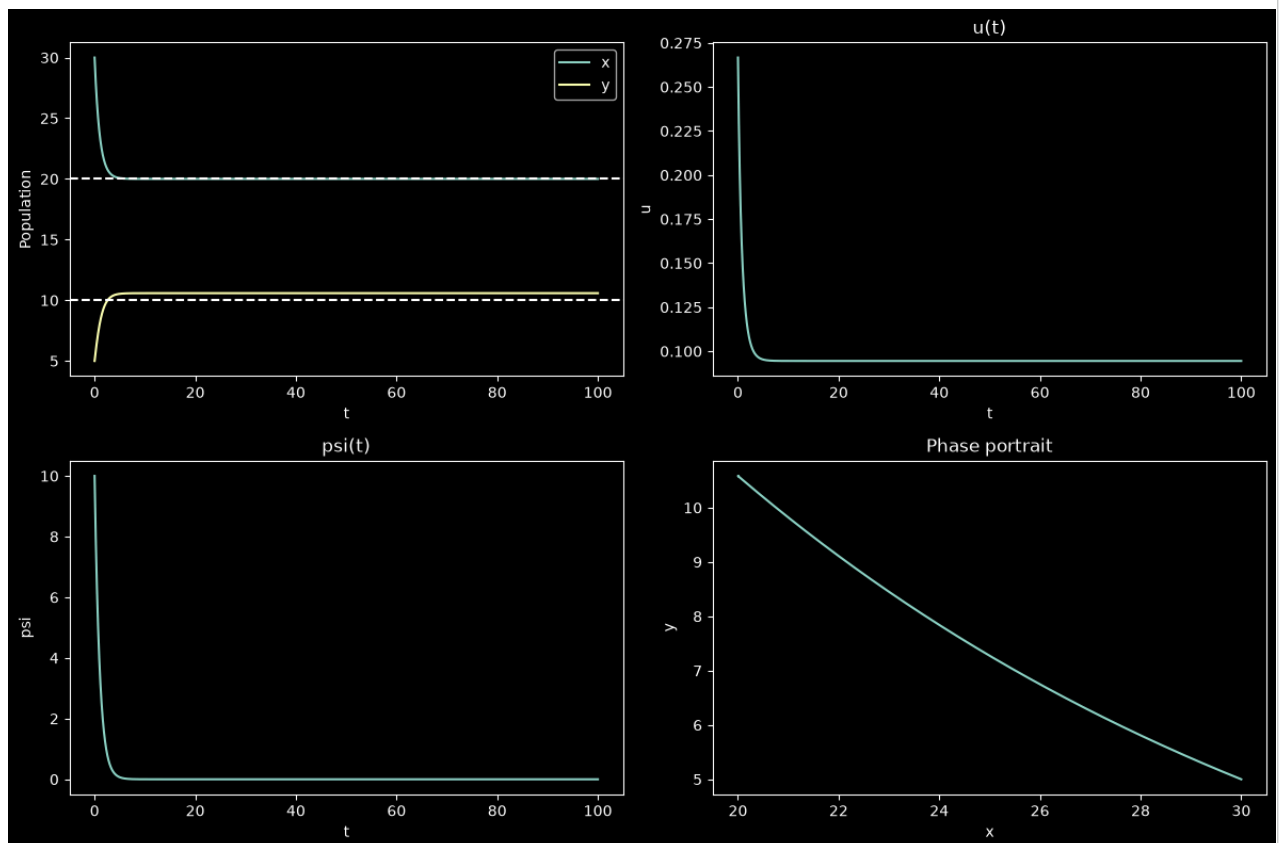
Метрики эксперимента:

```

Время сходимости ( $\tau$ ):      4.610461
Максимальное  $|\psi|$ :          10.000000
Максимальное  $|u|$ :            0.266667
Остаточная ошибка:          3.098367e-07

```

```
plot_experiment(result_mul)
```



Анализ результатов

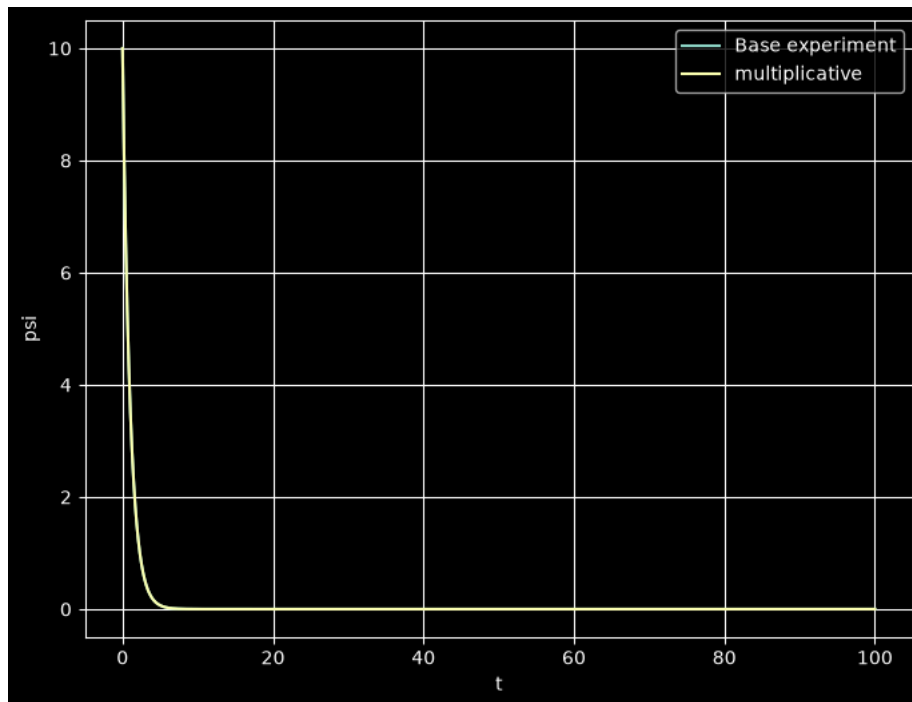
Полученные результаты практически совпадают с экспериментом для аддитивного управления. Система за то же время достигает положения равновесия, а остаточная ошибка остаётся пренебрежимо малой.

Основное отличие наблюдается только в величине управляющего воздействия, что связано с различным способом его формирования в аддитивном и мультипликативном законах управления.

Более подробное сравнение обоих вариантов управления будет выполнено в следующем разделе.

✓ Сравнение аддитивного и мультипликативного управления

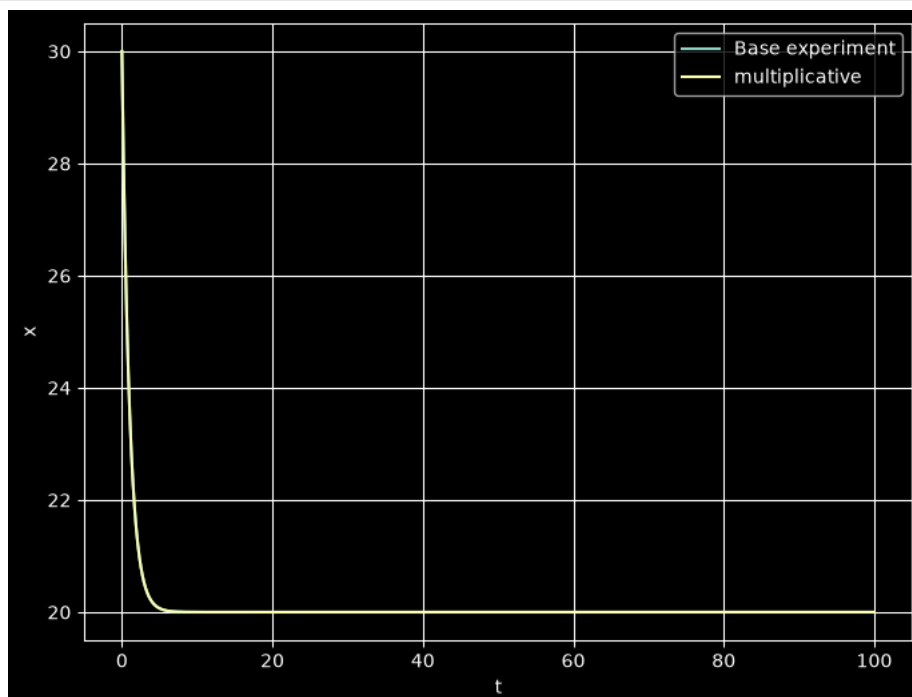
```
plot_compare(
    [result, result_mul],
    metric="psi"
)
```



Из графика видно, что кривые для аддитивного и мультипликативного управления полностью совпадают. Это означает, что в обоих случаях ошибка $\psi(t)$ уменьшается с одинаковой скоростью и система достигает положения равновесия за одно и то же время. Следовательно, с точки зрения динамики ошибки оба закона управления обеспечивают одинаковое качество регулирования.

✓ Сравнение изменения численности популяций $x(t)$

```
plot_compare(
    [result, result_mul],
    metric="x"
)
```

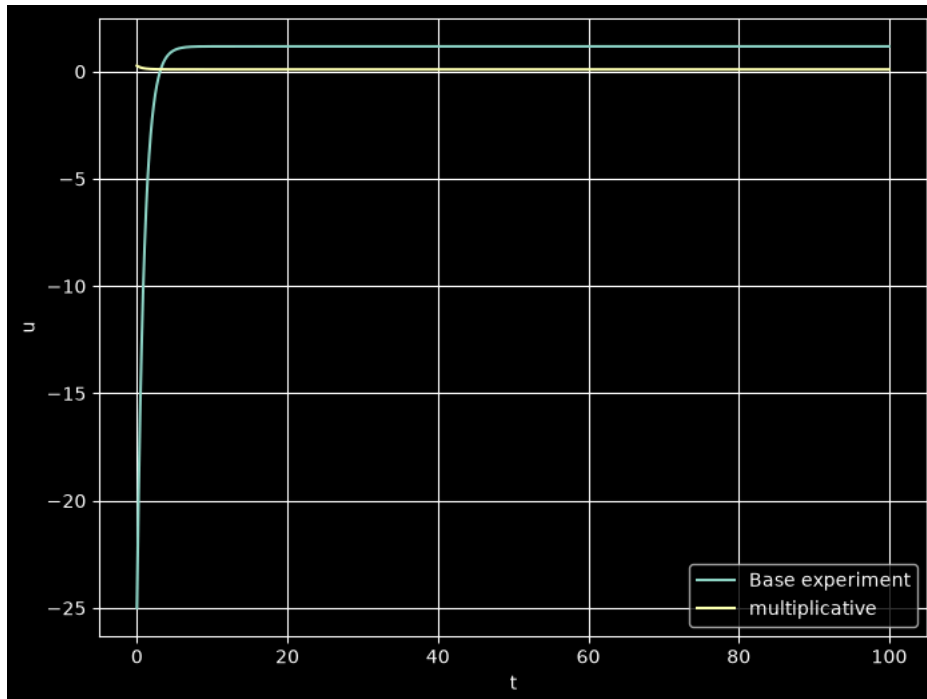


Из графика видно, что траектории изменения численности жертв для аддитивного и мультипликативного управления полностью совпадают. В обоих случаях популяция плавно уменьшается до равновесного значения $x^*=20$ без колебаний и

перерегулирования.

✓ Сравнение управляющего воздействия $u(t)$ и сводная таблица результатов

```
plot_compare(  
    [result, result_mul],  
    metric="u"  
)
```



```
print('Сводная таблица результатов')  
summary_table([  
    result,  
    result_mul  
)
```

Сводная таблица результатов

	name	T	tau	max_psi	max_u	e_final
0	Base experiment	1.0	4.610461	10.0	25.000000	3.098370e-07
1	multiplicative	1.0	4.610461	10.0	0.266667	3.098367e-07

Проведённое сравнение показало, что аддитивный и мультипликативный законы управления обеспечивают практически одинаковое качество регулирования. В обоих случаях система достигает положения равновесия за одинаковое время, а значения максимальной и остаточной ошибки практически совпадают.

Основное различие заключается в характере управляющего воздействия. При аддитивном управлении в начальный момент времени требуется значительно большее по величине воздействие, тогда как при мультипликативном управлении управление остаётся существенно меньше. Это связано с тем, что в первом случае управление непосредственно добавляется в уравнение модели, а во втором изменяет коэффициент взаимодействия между популяциями.

✓ Сохранение и воспроизведение эксперимента

Фреймворк позволяет сохранить параметры эксперимента и вычисленные метрики в JSON-файл. При необходимости эксперимент может быть полностью воспроизведён по сохранённой конфигурации.

```
save_path = "results/experiment.json"  
  
save_experiment(  
    result,  
    save_path  
)  
  
print("Конфигурация эксперимента успешно сохранена.")  
  
print(f"\nФайл: \n{save_path}")
```

Конфигурация эксперимента успешно сохранена.

Файл:
results/experiment.json

```
from pathlib import Path

print("Содержимое папки results:\n")

for file in Path("results").glob("*"):
    print(file.name)
```

Содержимое папки results:

experiment.json

```
reproduced = load_and_reproduce(save_path)
```

```
print("Исходные метрики:\n")

print(f"Время сходимости ( $\tau$ ):      {result.metrics.tau:.6f}")
print(f"Максимальное  $|\psi|$ :      {result.metrics.max_psi:.6f}")
print(f"Максимальное  $|u|$ :      {result.metrics.max_u:.6f}")
print(f"Остаточная ошибка:      {result.metrics.e_final:.6e}")

print("\nМетрики эксперимента:")
print(f"Время сходимости ( $\tau$ ):      {result.metrics.tau:.6f}")
print(f"Максимальное  $|\psi|$ :      {result.metrics.max_psi:.6f}")
print(f"Максимальное  $|u|$ :      {result.metrics.max_u:.6f}")
print(f"Остаточная ошибка:      {result.metrics.e_final:.6e}")
```

Исходные метрики:

Время сходимости (τ):	4.610461
Максимальное $ \psi $:	10.000000
Максимальное $ u $:	25.000000
Остаточная ошибка:	3.098370e-07

Метрики эксперимента:

Время сходимости (τ):	4.610461
Максимальное $ \psi $:	10.000000
Максимальное $ u $:	25.000000
Остаточная ошибка:	3.098370e-07

```
same = (
    abs(result.metrics.tau - reproduced.metrics.tau) < 1e-8
    and
    abs(result.metrics.max_psi - reproduced.metrics.max_psi) < 1e-8
    and
    abs(result.metrics.max_u - reproduced.metrics.max_u) < 1e-8
    and
    abs(result.metrics.e_final - reproduced.metrics.e_final) < 1e-8
)

print(f"\nСовпадают: {same}")
```

Совпадают: True

После загрузки конфигурации из JSON-файла эксперимент был автоматически воспроизведён. Совпадение всех вычисленных метрик подтверждает, что для повторного выполнения эксперимента достаточно сохранить только конфигурацию и параметры модели. Большие массивы данных (траектории $x(t)$, $y(t)$, $u(t)$, $\psi(t)$) сохранять не требуется.

▼ Демонстрация расширяемости фреймворка

Одним из требований задания являлась возможность расширения фреймворка за счёт добавления новых моделей без изменения его основной архитектуры.

Для проверки этого требования была реализована модификация модели Лотки–Вольтерра с учётом внутривидовой конкуренции.

При этом изменения потребовались только в двух компонентах:

- методе `equilibrium()`, отвечающем за вычисление положения равновесия;
- функции `make_rhs()`, формирующей правую часть системы дифференциальных уравнений.

Остальные компоненты фреймворка, включая запуск эксперимента, численное моделирование, вычисление метрик, сохранение результатов и построение графиков, продолжили работать без каких-либо изменений.

```
competition_cfg = ExperimentConfig(
    name="Competition model",
    model=CompetitionModelParams(
        epsilon=0.02,
        mu=0.01
    )
)

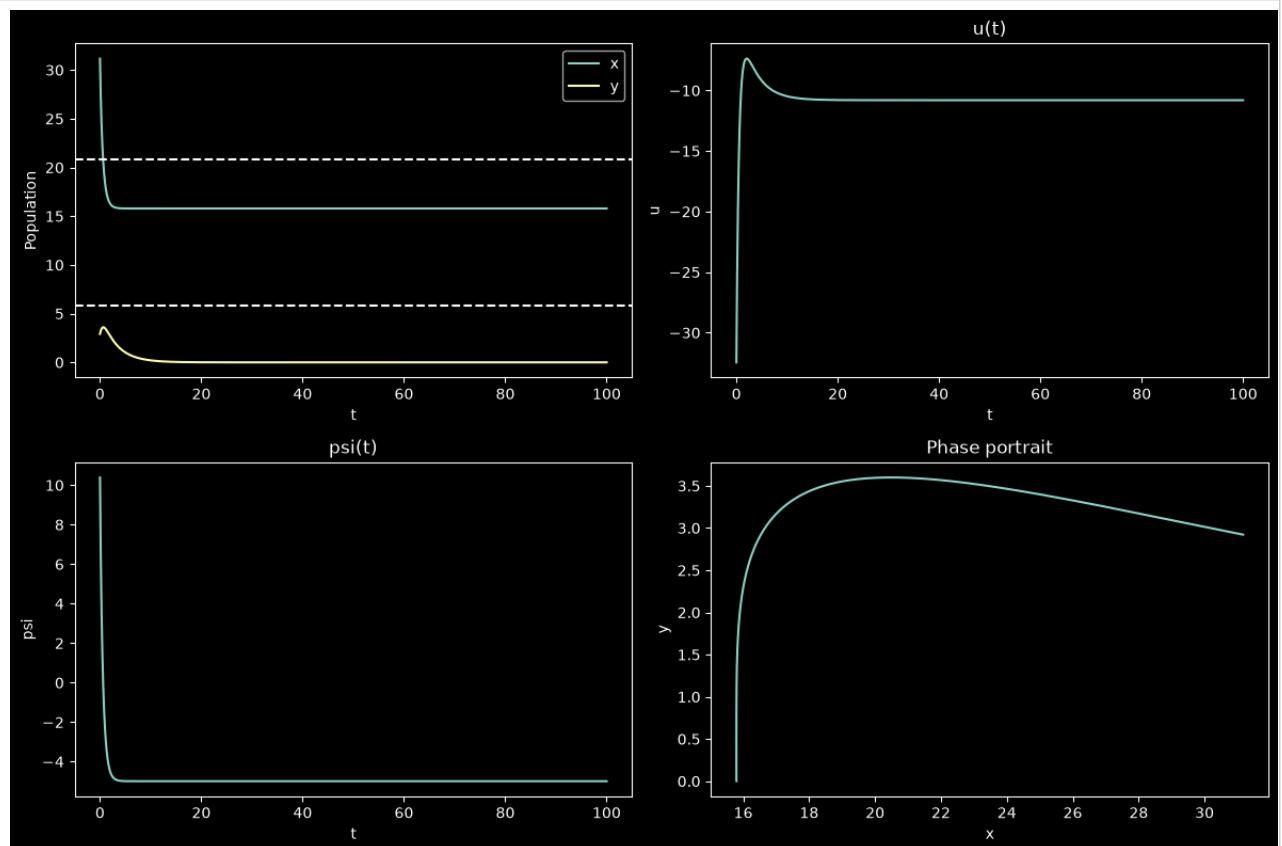
competition_result = run_experiment(
    competition_cfg
)

print("Метрики новой модели:\n")
print(f"Время сходимости ( $\tau$ ): {result.metrics.tau:.6f}")
print(f"Максимальное  $|\psi|$ : {result.metrics.max_psi:.6f}")
print(f"Максимальное  $|u|$ : {result.metrics.max_u:.6f}")
print(f"Остаточная ошибка: {result.metrics.e_final:.6e}")
```

Метрики новой модели:

Время сходимости (τ):	4.610461
Максимальное $ \psi $:	10.000000
Максимальное $ u $:	25.000000
Остаточная ошибка:	3.098370e-07

```
plot_experiment(
    competition_result
)
```



После добавления внутривидовой конкуренции поведение системы заметно изменилось по сравнению с классической моделью Лотки–Вольтерра. Популяция жертв быстро уменьшается и стабилизируется на уровне, отличном от расчетного равновесия, а численность хищников постепенно снижается практически до нуля.

График ошибки показывает, что после переходного процесса отклонение перестает изменяться и остается практически постоянным. Это связано с тем, что в рамках задания закон управления не изменялся и был подготовлен для классической модели Лотки–Вольтерра. После добавления членов внутривидовой конкуренции динамика системы изменилась, поэтому регулятор уже не обеспечивает точное достижение нового положения равновесия.

Управляющее воздействие имеет выраженный переходный процесс: в начале моделирования регулятор создает значительное воздействие, после чего управление быстро стабилизируется и далее поддерживается практически на постоянном уровне.

Фазовый портрет также существенно отличается от классической модели. Вместо движения к точке сосуществования двух популяций траектория приводит систему в режим, при котором численность жертв стабилизируется, а популяция хищников постепенно уменьшается почти до нуля.

Добавление внутривидовой конкуренции существенно изменяет динамику системы. При этом разработанный фреймворк не потребовал изменений в механизмах запуска экспериментов, вычисления метрик, сохранения результатов и построения графиков, что подтверждает его расширяемость. Полученный результат также показывает, что применение закона управления, синтезированного для классической модели, к модифицированной системе может приводить к появлению установившейся ошибки.

✓ Серия вычислительных экспериментов (аддитивное управление)

После проверки работы фреймворка проведем серию вычислительных экспериментов для исследования влияния параметра регулятора **T** на качество управления системой.

В ходе исследования параметр **T** изменяется в диапазоне от **0.1** до **5.0**. Для каждого значения автоматически выполняются:

- моделирование замкнутой системы;
- вычисление всех метрик качества управления;
- сохранение результатов эксперимента для последующего анализа и сравнения.

```
results = []

T_values = np.linspace(0.1, 5.0, 20)

Path("results").mkdir(exist_ok=True)

for T in T_values:

    cfg = ExperimentConfig(
        name=f"T={T:.2f}",
        control=ControlParams(T=T)
    )

    result = run_experiment(cfg)

    results.append(result)

print(f"Выполнено экспериментов: {len(results)}")
```

Выполнено экспериментов: 20

✓ Сохранение результатов

Все выполненные эксперименты сохраняются в отдельные JSON-файлы.

Дополнительно создается общий CSV-файл, содержащий вычисленные метрики всех экспериментов.

```
for result in results:

    filename = f"results/{result.config.name.replace('=', '_')}.json"

    save_experiment(
        result,
        filename
    )

save_metrics_csv(
    results,
    "results/all_metrics.csv"
)

print("Все эксперименты успешно сохранены.")
```

Все эксперименты успешно сохранены.

```
print("Содержимое папки results:\n")

for file in sorted(Path("results").glob("*")):
    print(file.name)
```

Содержимое папки results:

```
all_metrics.csv
experiment.json
T_0.10.json
T_0.36.json
T_0.62.json
T_0.87.json
T_1.13.json
T_1.39.json
T_1.65.json
T_1.91.json
T_2.16.json
T_2.42.json
T_2.68.json
T_2.94.json
T_3.19.json
T_3.45.json
T_3.71.json
T_3.97.json
T_4.23.json
T_4.48.json
T_4.74.json
T_5.00.json
```

Все результаты серии экспериментов успешно сохранены.

Для каждого значения параметра T сформирован отдельный JSON-файл с конфигурацией и вычисленными метриками.

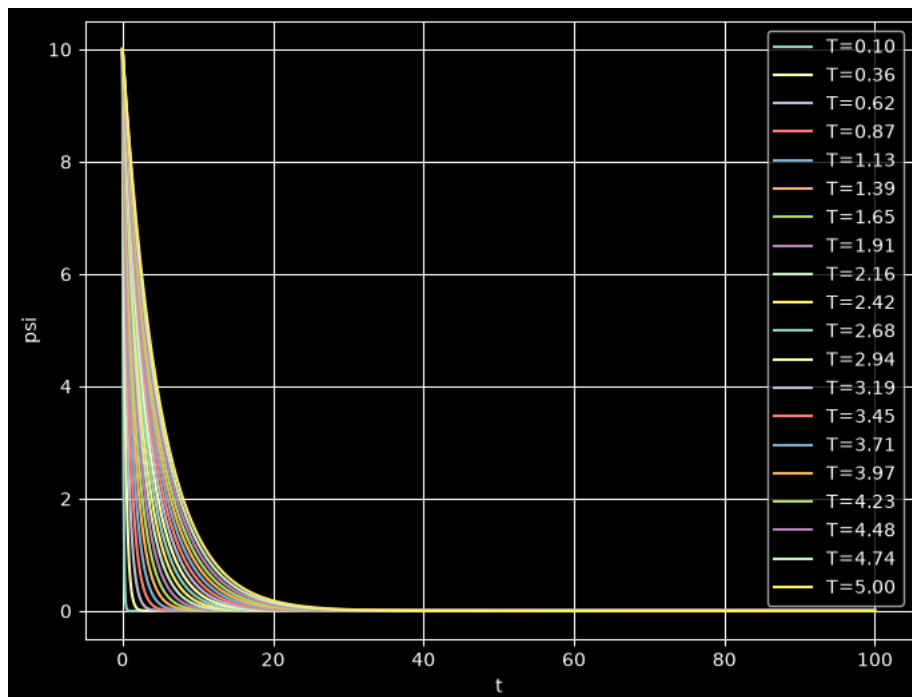
Кроме того, создан общий CSV-файл, позволяющий выполнять дальнейший анализ результатов.

Сравнение результатов

Макропеременная $\psi(t)$

Сравним изменение макропеременной $\psi(t)$ при различных значениях параметра T .

```
plot_compare(
    results,
    metric="psi"
)
```



Из графика видно, что параметр T существенно влияет на скорость уменьшения ошибки управления. При небольших значениях T ошибка $\psi(t)$ затухает значительно быстрее, поэтому система быстрее достигает положения равновесия.

С увеличением значения T переходный процесс становится более продолжительным: ошибка уменьшается медленнее, а время достижения равновесия возрастает. При этом характер изменения остаётся одинаковым для всех экспериментов — ошибка монотонно стремится к нулю без колебаний.

Таким образом, уменьшение параметра T позволяет ускорить работу системы, однако дальнейший анализ управляющего воздействия покажет, как это влияет на величину управления.

✓ Изменение популяции жертв $x(t)$

```
plot_compare(  
    results,  
    metric="x"  
)
```

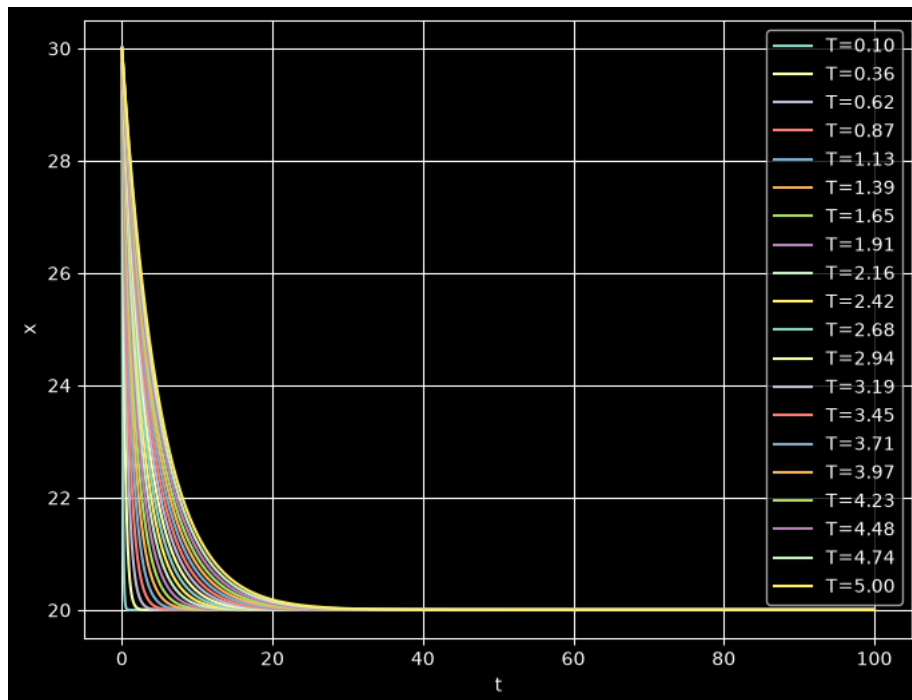


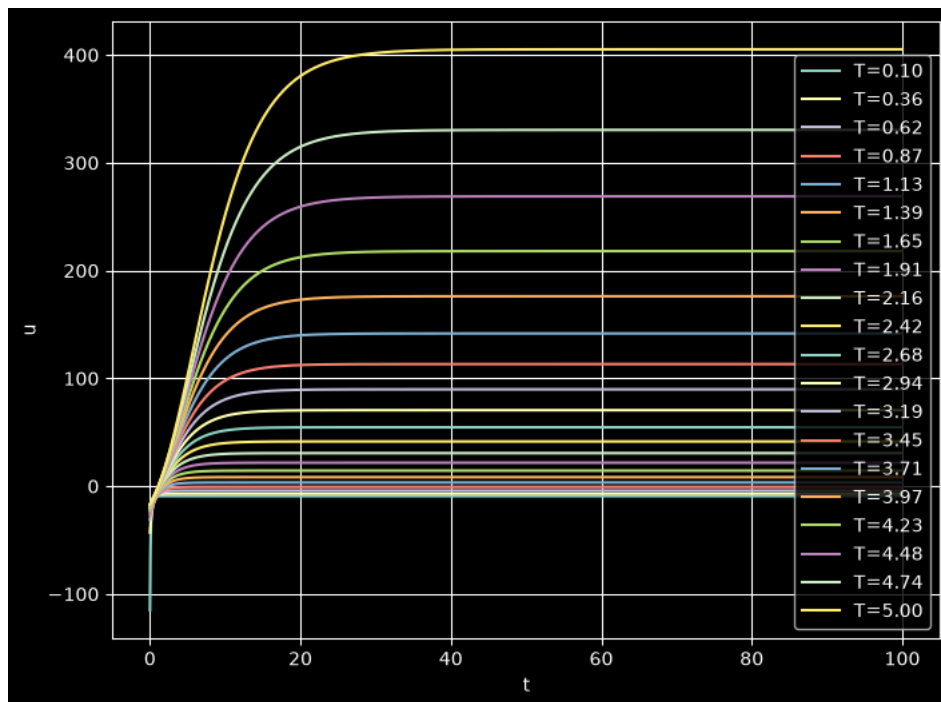
График показывает влияние параметра T на изменение численности жертв. Во всех экспериментах система сходится к одному и тому же равновесному значению $x^*=20$, однако скорость достижения этого состояния зависит от выбранного значения T .

При небольших значениях T численность жертв уменьшается значительно быстрее, поэтому переходный процесс завершается за меньшее время. С увеличением параметра T системе требуется больше времени для достижения равновесия.

При этом для всех рассмотренных значений T переходный процесс протекает плавно, без колебаний.

✓ Управляющее воздействие $u(t)$

```
plot_compare(  
    results,  
    metric="u"  
)
```



Из графика видно, что параметр T оказывает заметное влияние на величину управляющего воздействия. При малых значениях T управление быстро выходит на небольшой установившийся уровень. С увеличением параметра T величина управляющего воздействия возрастает, а переходный процесс становится более продолжительным.

Во всех рассмотренных экспериментах наблюдается одинаковый характер изменения управления: после начального переходного процесса значение $u(t)$ постепенно стабилизируется.

Предположительно, такое поведение связано с тем, что в рассматриваемой реализации управление формируется только для стабилизации переменной x_1 . При больших значениях параметра T система дольше находится в переходном режиме, вследствие чего изменяется состояние второй переменной модели, что приводит к увеличению требуемого управляющего воздействия.

Сводная таблица результатов

```
summary_table(results)
```

	name	T	tau	max_psi	max_u	e_final
0	T=0.10	0.100000	0.470047	10.0	115.000000	1.164453e-07
1	T=0.36	0.357895	1.650165	10.0	42.941176	4.290159e-07
2	T=0.62	0.615789	2.840284	10.0	31.239316	3.172020e-07
3	T=0.87	0.873684	4.030403	10.0	26.445783	1.413423e-07
4	T=1.13	1.131579	5.220522	10.0	23.837209	7.467875e-07
5	T=1.39	1.389474	6.400640	10.0	22.196970	1.306602e-07
6	T=1.65	1.647368	7.590759	10.0	21.070288	4.996629e-08
7	T=1.91	1.905263	8.780878	10.0	21.743033	2.526228e-07
8	T=2.16	2.163158	9.970997	10.0	30.650728	7.191698e-08
9	T=2.42	2.421053	11.151115	10.0	41.459269	2.240393e-07
10	T=2.68	2.678947	12.341234	10.0	54.574282	7.067412e-08
11	T=2.94	2.936842	13.531353	10.0	70.487956	4.168185e-08
12	T=3.19	3.194737	14.721472	10.0	89.797504	2.036376e-08
13	T=3.45	3.452632	15.901590	10.0	113.227589	1.368666e-07
14	T=3.71	3.710526	17.091709	10.0	141.657499	4.356216e-08
15	T=3.97	3.968421	18.281828	10.0	176.154172	1.672222e-08
16	T=4.23	4.226316	19.471947	10.0	218.012214	8.041575e-09
17	T=4.48	4.484211	20.652065	10.0	268.802494	5.635247e-09
18	T=4.74	4.742105	21.842184	10.0	330.431095	9.063285e-09
19	T=5.00	5.000000	23.032303	10.0	405.210842	2.301864e-08

Из сводной таблицы видно, что увеличение параметра T приводит к увеличению времени сходимости системы. Так, при $T = 0.1$ время сходимости составляет около **0.47**, а при $T = 5.0$ возрастает до **23.03**. Это подтверждает, что при больших значениях T переходный процесс протекает медленнее.

Максимальная ошибка во всех экспериментах остаётся одинаковой и равна **10**, поскольку система во всех случаях запускается из одного и того же начального состояния. Остаточная ошибка также имеет порядок 10^{-7} – 10^{-9} , что свидетельствует о высокой точности численного решения и успешном достижении заданного режима.

Интересно, что зависимость максимального управляющего воздействия от параметра T имеет нелинейный характер. При увеличении T значение $\max|u|$ сначала уменьшается, достигая минимального значения при $T \approx 1.6$ – 1.9 , а затем начинает быстро

возрастать. Предположительно, это связано с особенностями выбранного закона управления, который стабилизирует только переменную x . При больших значениях T системе требуется больше времени для достижения равновесия, что приводит к увеличению максимального управляющего воздействия.

```
best = next(r for r in results if r.config.name == "T=1.65")
```

```
print("Лучший эксперимент:")

print(f"Название:           {best.config.name}")
print(f"T:                  {best.config.control.T:.2f}")
print(f"Время сходимости:     {best.metrics.tau:.6f}")
print(f"Максимальное |ψ|:       {best.metrics.max_psi:.6f}")
print(f"Максимальное |u|:       {best.metrics.max_u:.6f}")
print(f"Остаточная ошибка:     {best.metrics.e_final:.6e}")
```

Лучший эксперимент:	
Название:	T=1.65
T:	1.65
Время сходимости:	7.590759
Максимальное ψ :	10.000000
Максимальное u :	21.070288
Остаточная ошибка:	4.996629e-08

Наиболее предпочтительным оказался эксперимент с $T=1.65$, поскольку при этом значении параметра регулятора достигается минимальная величина максимального управляющего воздействия при сохранении приемлемого времени сходимости и практически нулевой остаточной ошибки.

При $T=0.1$ система действительно сходится быстрее всего, однако для этого требуется значительно большее управляющее воздействие ($|u|_{\max}=115$). В реальных системах исполнительные механизмы имеют ограничения по мощности, поэтому столь большие значения управления нежелательны.

▼ Воспроизведение лучшего эксперимента

Проверим возможность полного воспроизведения эксперимента только по сохранённому JSON-файлу.

```
best_path = f"results/{best.config.name.replace(' ', '_')}.json"

print("Загружается файл:")

print(best_path)

best_loaded = load_and_reproduce(best_path)
```

Загружается файл:
results/T_1.65.json

```
print("Исходный эксперимент")
print(f"Время сходимости (τ):      {best.metrics.tau:.6f}")
print(f"Максимальное |ψ|:             {best.metrics.max_psi:.6f}")
print(f"Максимальное |u|:             {best.metrics.max_u:.6f}")
print(f"Остаточная ошибка:           {best.metrics.e_final:.6e}")

print("\nВоспроизведённый эксперимент")
print(f"Время сходимости (τ):      {best_loaded.metrics.tau:.6f}")
print(f"Максимальное |ψ|:           {best_loaded.metrics.max_psi:.6f}")
print(f"Максимальное |u|:           {best_loaded.metrics.max_u:.6f}")
print(f"Остаточная ошибка:         {best_loaded.metrics.e_final:.6e}")
```

Исходный эксперимент	
Время сходимости (τ):	7.590759
Максимальное ψ :	10.000000
Максимальное u :	21.070288
Остаточная ошибка:	4.996629e-08

Воспроизведённый эксперимент	
Время сходимости (τ):	7.590759
Максимальное ψ :	10.000000
Максимальное u :	21.070288
Остаточная ошибка:	4.996629e-08

```
print("\nСовпадение метрик\n")

print(
```



```

abs(best.metrics.tau - best_loaded.metrics.tau) < 1e-8,
abs(best.metrics.max_psi - best_loaded.metrics.max_psi) < 1e-8,
abs(best.metrics.max_u - best_loaded.metrics.max_u) < 1e-8,
abs(best.metrics.e_final - best_loaded.metrics.e_final) < 1e-8
)

```

Совпадение метрик

True True True True

Эксперимент был успешно воспроизведён по сохранённой конфигурации.

Совпадение всех вычисленных метрик подтверждает корректность реализации функций сохранения и загрузки результатов, а также воспроизводимость вычислительных экспериментов.

✓ Символьный вывод закона управления с помощью SymPy

Помимо численного моделирования во фреймворке реализован универсальный модуль символьного вывода закона управления с использованием библиотеки **SymPy**.

Модуль принимает на вход:

- символьные уравнения модели;
- макропеременную $\psi(x,y)$;
- параметр регулятора T .

После этого автоматически выполняются следующие этапы:

1. вычисляется производная макропеременной $\dot{\psi}$;
2. формируется условие АКАР $\dot{\psi} = -\psi/T$;
3. выполняется решение полученного уравнения относительно управления u ;
4. формируется лямбда-функция для последующего численного вычисления управления.

```

# Объявление символьных переменных состояния
x, y = sp.symbols("x y")

alpha, beta = sp.symbols("alpha beta")
delta, gamma = sp.symbols("delta gamma")
u = sp.symbols("u")
T = sp.symbols("T")
x_star = sp.symbols("x_star")

# Правая часть первого уравнения модели:
# x' = αx - βxy + u
f = alpha * x - beta * x * y + u

# Правая часть второго уравнения модели:
# y' = δxy - γy
g = delta * x * y - gamma * y

# Выбранная макропеременная
psi = x - x_star

```

```

# Символьный вывод закона управления
print("Правая часть x:")
sp.pprint(f)
print("\nПравая часть y:")
sp.pprint(g)
print("\nМакропеременная ψ:")
sp.pprint(psi)

# Получение закона управления разработанным модулем
u_expr, u_func = derive_control_symbolic(
    x,
    y,
    f,
    g,
    psi,
    u,
    T
)

print("\nЗакон управления, полученный символьным модулем:")
sp.pprint(u_expr)

# Аналитическая формула

```

```

expected = (
    -(x - x_star) / T
    - alpha * x
    + beta * x * y
)

print("\nАналитическая формула:")
sp.pprint(expected)

# Сравнение результатов
print("\nСовпадает ли символьный вывод с аналитическим?")

print(
    sp.simplify(u_expr - expected) == 0
)

```

Правая часть $\dot{x}$:

$$\alpha \cdot x - \beta \cdot x \cdot y + u$$

Правая часть $\dot{y}$:

$$\delta \cdot x \cdot y - \gamma \cdot y$$

Макропеременная ψ :

$$x - x_{star}$$

Закон управления, полученный символьным модулем:

$$\frac{-T \cdot x \cdot (\alpha - \beta \cdot y) - x + x_{star}}{T}$$

Аналитическая формула:

$$-\alpha \cdot x + \beta \cdot x \cdot y + \frac{-x + x_{star}}{T}$$

Совпадает ли символьный вывод с аналитическим?

True

✓ Второй пример работы для $\psi = x/y - c^*$

```

# Символьный вывод для  $\psi = x/y - c^*$ 
c_star = sp.symbols("c_star")
psi2 = x / y - c_star

u_expr2, u_func2 = derive_control_symbolic(
    x,
    y,
    f,
    g,
    psi2,
    u,
    T
)

print("Макропеременная:")
sp.pprint(psi2)

print("\nЗакон управления, полученный символьным модулем:")
sp.pprint(u_expr2)

# Аналитическая формула (полученная вручную)
expected2 = (
    -alpha * x
    + beta * x * y
    + delta * x**2
    - gamma * x
    - x / T
    + c_star * y / T
)

print("\nАналитическая формула:")
sp.pprint(expected2)

print("\nСовпадает ли символьный вывод с аналитическим?")
print(sp.simplify(u_expr2 - expected2) == 0)

```

Макропеременная:

$$x - c_{star} + \frac{x}{y}$$

Закон управления, полученный символьным модулем:

$$\frac{-T \cdot x \cdot (\alpha - \beta \cdot y - \delta \cdot x + \gamma) + C_{star} \cdot y - x}{T}$$

Аналитическая формула:

$$-\alpha \cdot x + \beta \cdot x \cdot y + \delta \cdot x^2 - \gamma \cdot x + \frac{C_{star} \cdot y}{T} - \frac{x}{T}$$

Совпадает ли символьный вывод с аналитическим?

True

✎ Использование lambda-функции

```
import inspect

print("\nАргументы lambda-функции:")
print(inspect.signature(u_func))

# Численное вычисление управления
u_value = u_func(
    30.0, # x
    10.0, # y
    2.0, # T
    1.0, # alpha
    0.1, # beta
    20.0 # x_star
)

print("\nПример численного вычисления закона управления")

print(f"x      = {30.0}")
print(f"y      = {10.0}")
print(f"T      = {2.0}")
print(f"alpha   = {1.0}")
print(f"beta    = {0.1}")
print(f"x_star  = {20.0}")

print(f"\nПолученное значение управления:")
print(f"u = {u_value:.6f}")
```

Аргументы lambda-функции:
(x, y, T, alpha, beta, x_star)

Пример численного вычисления закона управления

x	= 30.0
y	= 10.0
T	= 2.0
alpha	= 1.0
beta	= 0.1
x_star	= 20.0

Полученное значение управления:
u = -5.000000

ЗАКЛЮЧЕНИЕ

В ходе выполнения практической работы был разработан программный фреймворк на языке Python для проведения вычислительных экспериментов с моделью Лотки–Вольтерра и АКАР-управлением.

В процессе работы были изучены принципы построения модели «хищник–жертва», рассмотрен метод аналитического конструирования агрегированных регуляторов и реализованы законы аддитивного и мультипликативного управления. Для численного решения системы дифференциальных уравнений использовалась функция `solve_ivp` библиотеки SciPy.

Разработанный фреймворк позволяет запускать эксперименты с различными параметрами без изменения программного кода, вычислять основные метрики качества управления, сохранять результаты в форматах JSON и CSV, воспроизводить ранее выполненные эксперименты, а также строить графики и сравнивать результаты нескольких запусков.

Для проверки расширяемости фреймворка была реализована дополнительная модель Лотки–Вольтерра с внутривидовой конкуренцией. Ее добавление потребовало изменения только компонентов, отвечающих за описание модели и формирование правой части системы, тогда как остальные модули продолжили работать без изменений.

В Jupyter Notebook была проведена серия вычислительных экспериментов для различных значений параметра регулятора (T). По результатам моделирования были рассчитаны показатели качества управления, построены сравнительные графики и определено значение параметра, обеспечивающее наиболее удачный баланс между скоростью сходимости системы и величиной управляющего воздействия.

Дополнительно был реализован модуль символьного вывода закона управления с использованием библиотеки SymPy. Он позволяет автоматически получать аналитическое выражение управляющего

воздействия для заданной модели и макропеременной, что упрощает добавление новых вариантов управления.

Таким образом, поставленная цель работы была достигнута. Разработанный фреймворк удовлетворяет требованиям задания, успешно проходит все предусмотренные тесты и может использоваться для дальнейших исследований систем управления на основе метода АКАР.

ПРИЛОЖЕНИЕ А

Код программы фреймворка

```
import json
import csv
import numpy as np
from scipy.integrate import solve_ivp
import matplotlib.pyplot as plt
from dataclasses import dataclass, asdict, field
from pathlib import Path
from datetime import datetime
from typing import Optional
import pandas as pd
import sympy as sp
```

```
#
```

```
# РАЗДЕЛ 1: Конфигурация эксперимента
```

```
#
```

```
@dataclass
```

```
class ModelParams:
```

```
    """Параметры математической модели Лотки–Вольтерра."""
```

```
    #  $\alpha$  — скорость роста популяции жертв
```

```
    alpha: float = 1.0
```

```
    #  $\beta$  — интенсивность взаимодействия жертвы и хищника
```

```
    beta: float = 0.1
```

```
    #  $\delta$  — эффективность превращения добычи в прирост хищников
```

```
    delta: float = 0.075
```

```
    #  $\gamma$  — естественная смертность хищников
```

```
    gamma: float = 1.5
```

```
    def equilibrium(self):
```

```
"""
```

Вычисляет ненулевую точку равновесия системы.

Для классической модели:

$$x^* = \gamma / \delta$$

$$y^* = \alpha / \beta$$

Именно относительно этой точки строится управление
и вычисляются метрики качества.

```
"""
```

```
return self.gamma / self.delta, self.alpha / self.beta
```

```
@dataclass
```

```
class ControlParams:
```

```
    """Параметры АКАР-регулятора."""
```

```
    # Постоянная времени T.
```

```
    # Чем меньше T, тем быстрее система стремится к равновесию, но тем больше  
    # величина управляющего воздействия.
```

```
    T: float = 1.0
```

```
    control_type: str = "additive"
```

```
@dataclass
```

```
class SimParams:
```

```
    """Параметры численного моделирования."""
```

```
    # Время моделирования
```

```
    t_max: float = 100.0
```

```
    # Количество точек, в которых сохраняется решение
```

```
    n_eval: int = 10000
```

```
    # Начальное значение x задается как доля от равновесия:
```

```
    #  $x_0 = x^* \cdot x0\_frac$ 
```

```
    x0_frac: float = 1.5
```

```
    # Аналогично для y:
```

```
    #  $y_0 = y^* \cdot y0\_frac$ 
```

```
y0_frac: float = 0.5
```

```
# Относительная точность метода solve_ivp
```

```
rtol: float = 1e-8
```

```
@dataclass
```

```
class ExperimentConfig:
```

```
    """
```

```
    Полная конфигурация одного вычислительного эксперимента.
```

```
    Объединяет:
```

- параметры модели;
- параметры регулятора;
- параметры моделирования.

```
    Благодаря этому эксперимент можно запустить одной командой,  
    сохранить в JSON и затем полностью воспроизвести.
```

```
    """
```

```
    name: str = "experiment"
```

```
    model: ModelParams = field(default_factory=ModelParams)
```

```
    control: ControlParams = field(default_factory=ControlParams)
```

```
    sim: SimParams = field(default_factory=SimParams)
```

```
#
```

```
# РАЗДЕЛ 2: Ядро — модель и управление
```

```
#
```

```
def compute_control(x, y, x_star, p: ModelParams, c: ControlParams) -> float:
```

```
    """
```


Вычисляет управление $u(x, y)$.

Аддитивное (u входит в $\dot{x} = \alpha x - \beta xy + u$):

$$u = -(x - x^*)/T - \alpha x + \beta xy$$

Мультипликативное (u заменяет β в $\dot{x} = \alpha x - u \cdot xy$):

$$u = (\alpha x + (x - x^*)/T) / (x \cdot y)$$

"""

Подсказка: используйте c.control_type для выбора типа

if c.control_type == "additive":

$$\text{return } -(x - x_star) / c.T - p.alpha * x + p.beta * x * y$$

elif c.control_type == "multiplicative":

$$\text{return } (p.alpha * x + (x - x_star) / c.T) / (x * y)$$

else:

raise ValueError(f"Неизвестный тип управления: {c.control_type}")

#

ДОПОЛНИТЕЛЬНАЯ МОДЕЛЬ (вариант Б)

Лотка–Вольтерра с внутривидовой конкуренцией

#

@dataclass

class CompetitionModelParams(ModelParams):

"""

Модель Лотки–Вольтерра с внутривидовой конкуренцией.

$$x' = \alpha x - \beta xy - \epsilon x^2$$

$$y' = \delta xy - \gamma y - \mu y^2$$

```
"""
```

```
epsilon: float = 0.02
```

```
mu: float = 0.01
```

```
def equilibrium(self):
```

```
    """
```

```
    Возвращает ненулевое равновесие модели  
    с внутривидовой конкуренцией.
```

```
    """
```

```
    denom = self.beta * self.delta + self.mu * self.epsilon
```

```
    x_star = (  
        self.mu * self.alpha +  
        self.beta * self.gamma  
    ) / denom
```

```
    y_star = (  
        self.delta * self.alpha -  
        self.gamma * self.epsilon  
    ) / denom
```

```
    return x_star, y_star
```

```
#
```

```
# Формирование правой части системы дифференциальных уравнений  
#
```

```
def make_rhs(p: ModelParams, c: ControlParams,
```

```

    x_star: float, y_star: float):
"""

```

Создает функцию `rhs(t, state)`, описывающую динамику системы.

Именно эту функцию затем вызывает `solve_ivp` на каждом шаге численного интегрирования.

`p` — параметры математической модели;

`c` — параметры регулятора;

`x_star`,

`y_star` — координаты точки равновесия.

```

"""

```

Вложенная функция `rhs` вычисляет производные dx/dt и dy/dt для текущего состояния системы.

```

def rhs(t, state):

```

```

    x, y = state

```

```

    u = compute_control(x, y, x_star, p, c)

```

```

    # Проверяем тип управления.

```

```

    if c.control_type == "additive":

```

```

        # Классическая модель

```

```

        if not isinstance(p, CompetitionModelParams):

```

```

            dx = (

```

```

                p.alpha * x

```

```

                - p.beta * x * y

```

```

                + u

```

```

            )

```

```

            dy = (

```

```

                p.delta * x * y

```

```

                - p.gamma * y

```

```

            )

```

```

        # Модель с внутривидовой конкуренцией

```

```

        else:

```

```

            dx = (

```

```

        p.alpha * x
        - p.beta * x * y
        - p.epsilon * x**2
        + u
    )
    dy = (
        p.delta * x * y
        - p.gamma * y
        - p.mu * y**2
    )

```

Проверяем второй тип управления.

elif c.control_type == "multiplicative":

Классическая модель

if not isinstance(p, CompetitionModelParams):

```

    dx = (
        p.alpha * x
        - u * x * y
    )

```

```

    dy = (
        p.delta * x * y
        - p.gamma * y
    )

```

Модель с внутривидовой конкуренцией

else:

```

    dx = (
        p.alpha * x
        - u * x * y
        - p.epsilon * x**2
    )

```

```

    dy = (
        p.delta * x * y
    )

```

```

        - p.gamma * y
        - p.mu * y**2
    )

```

```

else:

```

```

    raise ValueError(
        f"Неизвестный тип управления: {c.control_type}"
    )

```

```

# Возвращаем производные.

```

```

return [dx, dy]

```

```

return rhs

```

```

#

```

```

# РАЗДЕЛ 3: Метрики качества

```

```

#

```

```

@dataclass

```

```

class Metrics:

```

```

    """Метрики качества управления."""

```

```

    tau: Optional[float] = None # время сходимости

```

```

    max_psi: float = 0.0         # max|ψ(t)|

```

```

    max_u: float = 0.0          # max|u(t)|

```

```

    e_final: float = 0.0        # остаточная ошибка

```

```

def compute_metrics(sol, x_star: float,

```

```

    p: ModelParams,

```

```

    c: ControlParams) -> Metrics:

```

```

    """

```

Вычисляет основные метрики качества управления
по результатам численного моделирования.

sol — результат работы solve_ivp;
x_star — координата равновесия;
p — параметры модели;
c — параметры регулятора.
"""

Если численный метод завершился с ошибкой, дальнейшие вычисления
выполнять бессмысленно.

Возвращаем бесконечные значения всех метрик.

if not sol.success:

```
    return Metrics(
        tau=float("inf"),
        max_psi=float("inf"),
        max_u=float("inf"),
        e_final=float("inf")
    )
```

Получаем решение системы.

sol.y содержит массив решений:

sol.y[0] — значения $x(t)$,

sol.y[1] — значения $y(t)$.

x_sol = sol.y[0]

y_sol = sol.y[1]

$\psi = x - x^*$

psi = x_sol - x_star

Для каждого момента времени вычисляем управление u .

compute_control вызывается для каждой пары (x, y) .

zip объединяет соответствующие элементы x_sol и y_sol.

```
u_sol = np.array([
    compute_control(x, y, x_star, p, c)
```

```

        for x, y in zip(x_sol, y_sol)
    ])

    # Максимальное отклонение системы от равновесия.
    max_psi = np.max(np.abs(psi))

    # Максимальная величина управляющего воздействия.
    max_u = np.max(np.abs(u_sol))

    # Остаточная ошибка в конце моделирования.
    # Берем последнее значение  $\psi$ .
    e_final = abs(psi[-1])

    # Порог окончания переходного процесса.  $|\psi|$  станет меньше 1% от начального
отклонения.
    threshold = 0.01 * abs(psi[0])

    idx = np.where(np.abs(psi) < threshold)[0]

    # Если такие моменты существуют,
    # временем сходимости считаем первый из них.
    if len(idx) > 0:
        tau = sol.t[idx[0]]

    # Иначе считаем, что за время моделирования система не сошлась.
    else:
        tau = None

    return Metrics(
        tau=tau,
        max_psi=max_psi,
        max_u=max_u,
        e_final=e_final
    )

```

#

РАЗДЕЛ 4: Запуск эксперимента

#

@dataclass

class ExperimentResult:

"""Результат эксперимента."""

config: ExperimentConfig

metrics: Metrics

t: np.ndarray

x: np.ndarray

y: np.ndarray

u: np.ndarray

psi: np.ndarray

timestamp: str = field(default_factory=lambda: datetime.now().isoformat())

def run_experiment(config: ExperimentConfig) -> ExperimentResult:

"""

Запускает эксперимент и возвращает ExperimentResult.

Это основная функция фреймворка.

Пример использования:

cfg = ExperimentConfig(name="test", control=ControlParams(T=0.5))

result = run_experiment(cfg)

"""

p = config.model

c = config.control

s = config.sim


```

# Находим точку равновесия модели.
x_star, y_star = p.equilibrium()

# Вычисляем начальные условия.
# Они задаются как доля от точки равновесия.
x0 = x_star * s.x0_frac
y0 = y_star * s.y0_frac

# Формируем массив времени, в которых будет сохраняться решение.
t_eval = np.linspace(0, s.t_max, s.n_eval)

#Правая часть
rhs = make_rhs(p, c, x_star, y_star)
#решение дифф уравнений
sol = solve_ivp(rhs, (0, s.t_max), [x0, y0],
                t_eval=t_eval, method='RK45', rtol=s.rtol)

# Из результата извлекаем найденные траектории и вычисляем управление,
метрики
x_sol = sol.y[0]
y_sol = sol.y[1]
u_sol = np.array([compute_control(x, y, x_star, p, c)
                  for x, y in zip(x_sol, y_sol)])
psi_sol = x_sol - x_star

metrics = compute_metrics(sol, x_star, p, c)

return ExperimentResult(
    config=config, metrics=metrics,
    t=sol.t, x=x_sol, y=y_sol, u=u_sol, psi=psi_sol
)

#

```

РАЗДЕЛ 5: Сохранение и загрузка

#

```
def save_experiment(result: ExperimentResult, path: str) -> None:
```

```
    """
```

```
        Сохраняет конфигурацию и метрики в JSON-файл.
```

```
        Массивы t, x, y, u, psi НЕ сохраняются (слишком большие).
```

```
        Для воспроизведения достаточно конфигурации.
```

```
        Формат файла:
```

```
        {
            "name": "...",
            "timestamp": "...",
            "config": { ... },
            "metrics": { ... }
        }
    """
```

```
    data = {
        "name": result.config.name,
        "timestamp": result.timestamp,
        "config": asdict(result.config),
        "metrics": asdict(result.metrics)
    }
```

```
    # Если папка для сохранения не существует, создаем ее автоматически.
```

```
    Path(path).parent.mkdir(parents=True, exist_ok=True)
```

```
    with open(path, "w", encoding="utf-8") as f:
```

```
        json.dump(data, f, indent=4)
```

```
def load_and_reproduce(path: str) -> ExperimentResult:
```

```
    """
```

```
        Загружает конфигурацию из JSON и воспроизводит эксперимент.
```

Возвращает ExperimentResult как при обычном запуске.

```
"""
```

```
# TODO: реализуйте загрузку и воспроизведение
```

```
with open(path, "r", encoding="utf-8") as f:
```

```
    data = json.load(f)
```

```
cfg = data["config"]
```

```
config = ExperimentConfig(
```

```
    name=cfg["name"],
```

```
    model=ModelParams(**cfg["model"]),
```

```
    control=ControlParams(**cfg["control"]),
```

```
    sim=SimParams(**cfg["sim"])
```

```
)
```

```
return run_experiment(config)
```

```
def save_metrics_csv(results: list, path: str) -> None:
```

```
"""
```

Сохраняет метрики нескольких экспериментов в CSV.

Каждая строка: name, T, alpha, beta, delta, gamma,

tau, max_psi, max_u, e_final

```
"""
```

```
# TODO: реализуйте сохранение в CSV
```

```
Path(path).parent.mkdir(parents=True, exist_ok=True)
```

```
with open(path,
```

```
    "w",
```

```
    newline="",
```

```
    encoding="utf-8") as f:
```

```
writer = csv.writer(f)
```

```
writer.writerow([
```

```
    "name",
```

```
    "T",
```

```
        "alpha",
        "beta",
        "delta",
        "gamma",
        "tau",
        "max_psi",
        "max_u",
        "e_final"
    ])
```

for result in results:

```
    cfg = result.config
    m = result.metrics
```

```
    writer.writerow([
        cfg.name,
        cfg.control.T,
        cfg.model.alpha,
        cfg.model.beta,
        cfg.model.delta,
        cfg.model.gamma,
        m.tau,
        m.max_psi,
        m.max_u,
        m.e_final
    ])
```

```
#
```

```
# РАЗДЕЛ 6: Визуализация
```

```
#
```

```

def plot_experiment(result: ExperimentResult,
                    save_path: Optional[str] = None) -> None:
    """
    Строит стандартные 4 графика для одного эксперимента:
    -  $x(t)$  и  $y(t)$  с линиями равновесия
    -  $u(t)$ 
    -  $\psi(t)$ 
    - фазовый портрет  $(x, y)$ 

    Если save_path задан — сохраняет PNG, иначе показывает plt.show().
    """
    fig, axs = plt.subplots(2, 2, figsize=(12, 8))

    x_star, y_star = result.config.model.equilibrium()

    axs[0, 0].plot(result.t, result.x, label="x")
    axs[0, 0].plot(result.t, result.y, label="y")
    axs[0, 0].axhline(x_star, linestyle="--")
    axs[0, 0].axhline(y_star, linestyle="--")
    axs[0, 0].set_xlabel("t")
    axs[0, 0].set_ylabel("Population")
    axs[0, 0].legend()

    axs[0, 1].plot(result.t, result.u)
    axs[0, 1].set_title("u(t)")
    axs[0, 1].set_xlabel("t")
    axs[0, 1].set_ylabel("u")

    axs[1, 0].plot(result.t, result.psi)
    axs[1, 0].set_title("psi(t)")
    axs[1, 0].set_xlabel("t")
    axs[1, 0].set_ylabel("psi")

```

```

    axs[1, 1].plot(result.x, result.y)
    axs[1, 1].set_title("Phase portrait")
    axs[1, 1].set_xlabel("x")
    axs[1, 1].set_ylabel("y")

```

```

plt.tight_layout()

```

```

if save_path:
    Path(save_path).parent.mkdir(parents=True, exist_ok=True)
    plt.savefig(save_path)
else:
    plt.show()

```

```

plt.close()

```

```

def plot_compare(results: list, metric: str = "psi",
                 save_path: Optional[str] = None) -> None:

```

```

    """

```

Сравнивает несколько экспериментов на одном графике.

metric: "psi" — график $\psi(t)$, "x" — график $x(t)$, "u" — график $u(t)$

Каждый эксперимент подписывается именем из config.name.

Пример использования:

```

    r1 = run_experiment(ExperimentConfig("T=0.5",
control=ControlParams(T=0.5)))
    r2 = run_experiment(ExperimentConfig("T=1.0",
control=ControlParams(T=1.0)))
    r3 = run_experiment(ExperimentConfig("T=2.0",
control=ControlParams(T=2.0)))

```

```

    plot_compare([r1, r2, r3], metric="psi")

```

```

    """

```

```

plt.figure(figsize=(8, 6))

```

```
for result in results:
```

```
    if metric == "psi":
```

```
        y = result.psi
```

```
    elif metric == "x":
```

```
        y = result.x
```

```
    elif metric == "u":
```

```
        y = result.u
```

```
    else:
```

```
        raise ValueError("Неизвестная метрика")
```

```
plt.plot(result.t, y, label=result.config.name)
```

```
plt.xlabel("t")
```

```
plt.ylabel(metric)
```

```
plt.legend()
```

```
plt.grid(True)
```

```
if save_path:
```

```
    Path(save_path).parent.mkdir(parents=True, exist_ok=True)
```

```
    plt.savefig(save_path)
```

```
else:
```

```
    plt.show()
```

```
plt.close()
```

```
def summary_table(results: list) -> None:
```

```
    """
```

```
    Выводит сводную таблицу метрик для списка экспериментов.
```

```
    Формат вывода — pandas DataFrame.
```

```
    Колонки: name, T, tau, max_psi, max_u, e_final
```

```
    """
```

```
    # TODO: реализуйте сводную таблицу
```

```

rows = []

for result in results:
    rows.append({
        "name": result.config.name,
        "T": result.config.control.T,
        "tau": result.metrics.tau,
        "max_psi": result.metrics.max_psi,
        "max_u": result.metrics.max_u,
        "e_final": result.metrics.e_final
    })

df = pd.DataFrame(rows)

print(df)

```

```
#
```

```
# УНИВЕРСАЛЬНЫЙ СИМВОЛЬНЫЙ ВЫВОД АКАР
```

```
#
```

```
def derive_control_symbolic(x, y, f, g, psi, u, T):
    """
    Автоматически выводит закон управления АКАР
    средствами библиотеки SymPy.

```

На вход передаются:

x, y — символьные переменные состояния;
 f — правая часть первого уравнения $x' = f(\dots)$;
 g — правая часть второго уравнения $y' = g(\dots)$;
 ψ — выбранная макропеременная $\psi(x, y)$;

u — символ управления;
T — постоянная времени АКАР.

Возвращает:

u_expr — символьную формулу управления;
u_func — функцию для численного вычисления управления.

"""

Вычисляем производную макропеременной ψ
по правилу полной производной: $\dot{\psi} = \partial\psi/\partial x \cdot \dot{x} + \partial\psi/\partial y \cdot \dot{y}$
Вместо $\dot{x}$ и $\dot{y}$ используются заданные функции $f(x,y,u)$ и $g(x,y,u)$.

```
psi_dot = (  
    sp.diff(psi, x) * f +  
    sp.diff(psi, y) * g  
)
```

Упрощаем полученное выражение.

```
psi_dot = sp.simplify(psi_dot)
```

Формируем условие АКАР: $\dot{\psi} = -\psi / T$

Eq() создает символьное уравнение.

```
equation = sp.Eq(  
    psi_dot,  
    -psi / T  
)
```

Решаем полученное уравнение относительно управления u.

solve() возвращает список решений, поэтому берем первое.

```
u_expr = sp.solve(  
    equation,  
    u  
) [0]
```

```
u_expr = sp.simplify(u_expr)
```

```

# Определяем, какие параметры присутствуют в формуле.
# Исключаем x и y, так как они являются переменными состояния.
parameters = sorted(
    list(
        u_expr.free_symbols - {x, y}
    ),
    key=lambda s: s.name
)

u_func = sp.lambdify(
    (x, y, *parameters),
    u_expr,
    "numpy"
)

# Возвращаем: символьную формулу; функцию для вычислений.
return u_expr, u_func

```

```

#

```

```

# РАЗДЕЛ 7: Тесты — НЕ МЕНЯТЬ

```

```

#

```

```

# Все тесты должны пройти без ошибок. Запуск: python framework_skeleton.py

```

```

def run_tests():

```

```

print("Запуск тестов...")
errors = []

# Тест 1: равновесие
p = ModelParams()
xs, ys = p.equilibrium()
assert abs(xs - 20.0) < 1e-6 and abs(ys - 10.0) < 1e-6, "Тест 1 провален:
equilibrium()"
print(" [OK] Тест 1: equilibrium()")

# Тест 2: управление не падает
try:
    c = ControlParams(T=1.0, control_type="additive")
    u = compute_control(xs*1.5, ys*0.5, xs, p, c)
    assert isinstance(u, (int, float)), "Тест 2: compute_control должен возвращать
число"

    print(" [OK] Тест 2: compute_control (additive)")
except Exception as e:
    print(f" [FAIL] Тест 2: {e}")
    errors.append(2)

# Тест 3: мультипликативное управление
try:
    c2 = ControlParams(T=1.0, control_type="multiplicative")
    u2 = compute_control(xs*1.5, ys*0.5, xs, p, c2)
    assert isinstance(u2, (int, float))
    print(" [OK] Тест 3: compute_control (multiplicative)")
except Exception as e:
    print(f" [FAIL] Тест 3: {e}")
    errors.append(3)

# Тест 4: run_experiment
try:
    cfg = ExperimentConfig(name="test")
    r = run_experiment(cfg)

```

```
    assert len(r.t) > 0 and len(r.x) == len(r.t)
    print(" [OK] Тест 4: run_experiment()")
except Exception as e:
    print(f" [FAIL] Тест 4: {e}")
    errors.append(4)
```

Тест 5: метрики

```
try:
    assert r.metrics.max_psi > 0
    assert r.metrics.max_u > 0
    assert r.metrics.e_final < 1.0
    print(" [OK] Тест 5: метрики разумные")
except Exception as e:
    print(f" [FAIL] Тест 5: {e}")
    errors.append(5)
```

Тест 6: сохранение и воспроизведение

```
try:
    save_experiment(r, "/tmp/test_exp.json")
    r2 = load_and_reproduce("/tmp/test_exp.json")
    assert abs(r2.metrics.tau - r.metrics.tau) < 1e-3
    print(" [OK] Тест 6: save/load/reproduce")
except Exception as e:
    print(f" [FAIL] Тест 6: {e}")
    errors.append(6)
```

Тест 7: CSV

```
try:
    cfg2 = ExperimentConfig(name="test2", control=ControlParams(T=2.0))
    r3 = run_experiment(cfg2)
    save_metrics_csv([r, r3], "/tmp/test_metrics.csv")
    with open("/tmp/test_metrics.csv") as f:
        lines = f.readlines()
    assert len(lines) == 3 # заголовок + 2 строки
    print(" [OK] Тест 7: save_metrics_csv()")
```

```

except Exception as e:
    print(f" [FAIL] Тест 7: {e}")
    errors.append(7)

# Тест 8: plot_experiment не падает
try:
    plot_experiment(r, save_path="/tmp/test_plot.png")
    assert Path("/tmp/test_plot.png").exists()
    print(" [OK] Тест 8: plot_experiment()")
except Exception as e:
    print(f" [FAIL] Тест 8: {e}")
    errors.append(8)

# Тест 9: plot_compare не падает
try:
    plot_compare([r, r3], metric="psi", save_path="/tmp/test_compare.png")
    assert Path("/tmp/test_compare.png").exists()
    print(" [OK] Тест 9: plot_compare()")
except Exception as e:
    print(f" [FAIL] Тест 9: {e}")
    errors.append(9)

# Тест 10: summary_table не падает
try:
    summary_table([r, r3])
    print(" [OK] Тест 10: summary_table()")
except Exception as e:
    print(f" [FAIL] Тест 10: {e}")
    errors.append(10)

print()
if not errors:
    print("Все тесты пройдены!")
else:
    print(f"Провалено тестов: {len(errors)} (тесты {errors})")

```

```
return len(errors) == 0
```

```
if __name__ == "__main__":  
    run_tests()
```